

```
// Merged files from branch: main

// --- Start of pubspec.yaml ---
name: cl837_accelerometer
description: CL837 Accelerometer data reader
publish_to: 'none'
version: 1.0.0+1

environment:
  sdk: '>=3.0.0 <4.0.0'

dependencies:
  flutter:
    sdk: flutter
  flutter_blue_plus: ^1.31.8
  permission_handler: ^11.0.1
  intl: ^0.18.0
  cupertino_icons: ^1.0.2

dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^2.0.0

flutter:
  uses-material-design: true
// --- End of pubspec.yaml ---

// --- Start of lib\accelerometer_service.dart ---
import 'dart:async';
import 'package:flutter/foundation.dart';
import 'package:flutter_blue_plus/flutter_blue_plus.dart';

class SensorData {
  final DateTime timestamp;
  final double x;
  final double y;
  final double z;
  final int? heartRate;
  final int? batteryLevel;

  const SensorData({
    required this.timestamp,
    required this.x,
    required this.y,
    required this.z,
    this.heartRate,
    this.batteryLevel,
  });
}

class SensorService {
  // Accelerometer Service & Characteristics
  static const String _accelServiceUuid =

```

```

'aae28f00-71b5-42a1-8c3c-f9cf6ac969d0';
    static const String _accelDataCharUuid =
'aae28f01-71b5-42a1-8c3c-f9cf6ac969d0';

    // Heart Rate Service & Characteristic
    static const String _heartRateServiceUuid =
'0000180d-0000-1000-8000-00805f9b34fb';
    static const String _heartRateCharUuid =
'00002a37-0000-1000-8000-00805f9b34fb';

    // Battery Service & Characteristic
    static const String _batteryServiceUuid =
'0000180f-0000-1000-8000-00805f9b34fb';
    static const String _batteryCharUuid = '00002a19-0000-1000-8000-00805f9b34fb';

    static const double _scaleFactor = 8.0 / 32768.0; // For  $\pm 8g$  range

    final _dataStreamController = StreamController<SensorData>.broadcast();
    final _stateStreamController = StreamController<bool>.broadcast();

    StreamSubscription? _accelSubscription;
    StreamSubscription? _heartRateSubscription;
    StreamSubscription? _batterySubscription;

    int? _lastHeartRate;
    int? _lastBatteryLevel;

    Stream<SensorData> get dataStream => _dataStreamController.stream;
    Stream<bool> get connectionStream => _stateStreamController.stream;

    void _processAccelData(List<int> value) {
        if (value.length < 3) return;
        try {
            if (value[0] != 0xFF) return;

            if (value[2] == 0x0c) {
                final now = DateTime.now();

                for (var i = 3; i < value.length - 1; i += 6) {
                    if (i + 5 >= value.length) break;

                    final rawX = _convertToSigned16(value[i] | (value[i + 1] << 8));
                    final rawY = _convertToSigned16(value[i + 2] | (value[i + 3] << 8));
                    final rawZ = _convertToSigned16(value[i + 4] | (value[i + 5] << 8));

                    final x = rawX * _scaleFactor;
                    final y = rawY * _scaleFactor;
                    final z = rawZ * _scaleFactor;

                    _dataStreamController.add(SensorData(
                        timestamp: now,
                        x: x,
                        y: y,
                        z: z,
                    ));
                }
            }
        } catch (e) {
            // Handle exception
        }
    }

```

```

        heartRate: _lastHeartRate,
        batteryLevel: _lastBatteryLevel,
    ));
    }
}
} catch (e) {
    debugPrint('Error processing accelerometer data: $e');
}
}

void _processHeartRate(List<int> value) {
    try {
        debugPrint('Raw heart rate data: ${value.map((b) =>
'0x${b.toRadixString(16).padLeft(2, '0')}}').join(', ')}');
        if (value.isEmpty) return;

        final flags = value[0];
        final isUint16 = (flags & 0x01) == 1;

        int heartRate;
        if (isUint16 && value.length >= 3) {
            heartRate = value[1] | (value[2] << 8);
        } else if (value.length >= 2) {
            heartRate = value[1];
        } else {
            return;
        }

        _lastHeartRate = heartRate;
        debugPrint('Processed Heart Rate: $_lastHeartRate BPM');
    } catch (e) {
        debugPrint('Error processing heart rate data: $e');
    }
}

void _processBattery(List<int> value) {
    try {
        debugPrint('Raw battery data: ${value.map((b) =>
'0x${b.toRadixString(16).padLeft(2, '0')}}').join(', ')}');
        if (value.isEmpty) return;

        _lastBatteryLevel = value[0];
        debugPrint('Processed Battery Level: $_lastBatteryLevel%');
    } catch (e) {
        debugPrint('Error processing battery data: $e');
    }
}

int _convertToSigned16(int value) {
    value &= 0xFFFF;
    return (value & 0x8000) != 0 ? -(0x10000 - value) : value;
}

```

```

Future<void> start(BluetoothDevice device) async {

```

```

try {
  debugPrint('\n=== STARTING BLE SERVICE SETUP ===');
  final services = await device.discoverServices();

  debugPrint('Found ${services.length} services:');
  for (var service in services) {
    debugPrint('Service: ${service.uuid}');
  }

  // Log all discovered services and their characteristics
  for (var service in services) {
    debugPrint('Service UUID: ${service.uuid}');
    for (var char in service.characteristics) {
      debugPrint('  Characteristic UUID: ${char.uuid}');
      debugPrint('  Properties: read=${char.properties.read}, '
        'write=${char.properties.write}, '
        'notify=${char.properties.notify}, '
        'indicate=${char.properties.indicate}');
    }
  }

  // Setup Accelerometer
  try {
    debugPrint('Setting up Accelerometer service...');
    final accelService = services.firstWhere(
      (s) => s.uuid.toString().toLowerCase() ==
        _accelServiceUuid.toLowerCase(),
      orElse: () => throw Exception('Accelerometer service not found'),
    );
    debugPrint('Found Accelerometer service: ${accelService.uuid}');

    final accelChar = accelService.characteristics.firstWhere(
      (c) => c.uuid.toString().toLowerCase() ==
        _accelDataCharUuid.toLowerCase(),
      orElse: () => throw Exception('Accelerometer characteristic not
found'),
    );
    debugPrint('Found Accelerometer characteristic: ${accelChar.uuid}');
    debugPrint('Accelerometer Properties: read=${accelChar.properties.read},
,
    'notify=${accelChar.properties.notify}');

    // Enable notifications
    final success = await accelChar.setNotifyValue(true);
    debugPrint('Accelerometer notifications enabled: $success');

    _accelSubscription = accelChar.lastValueStream.listen(
      (value) {
        debugPrint('Accelerometer data received: ${value.map((b) =>
'0x${b.toRadixString(16).padLeft(2, '0')}}').join(', ')}');
        _processAccelData(value);
      },
      onError: (error) {
        debugPrint('Accelerometer notification error: $error');
      }
    );
  }
}

```

```

        _stateStreamController.add(false);
    },
);
} catch (e) {
    debugPrint('Accelerometer setup error: $e');
}

// Setup Heart Rate
try {
    debugPrint('Setting up Heart Rate service...');
    final hrService = services.firstWhere(
        (s) => s.uuid.toString().toLowerCase() ==
        _heartRateServiceUuid.toLowerCase(),
        orElse: () => throw Exception('Heart rate service not found'),
    );
    debugPrint('Found Heart Rate service: ${hrService.uuid}');

    final hrChar = hrService.characteristics.firstWhere(
        (c) => c.uuid.toString().toLowerCase() ==
        _heartRateCharUuid.toLowerCase(),
        orElse: () => throw Exception('Heart rate characteristic not found'),
    );
    debugPrint('Found Heart Rate characteristic: ${hrChar.uuid}');
    debugPrint('Heart Rate Properties: read=${hrChar.properties.read}, '
        'notify=${hrChar.properties.notify}');

    // Try initial read
    if (hrChar.properties.read) {
        try {
            final value = await hrChar.read();
            debugPrint('Initial heart rate read: ${value.map((b) =>
                '0x${b.toRadixString(16).padLeft(2, '0')}}').join(', ')}');
            _processHeartRate(value);
        } catch (e) {
            debugPrint('Error reading initial heart rate: $e');
        }
    }

    // Enable notifications
    if (hrChar.properties.notify) {
        try {
            final success = await hrChar.setNotifyValue(true);
            debugPrint('Heart Rate notifications enabled: $success');
            _heartRateSubscription = hrChar.lastValueStream.listen(
                (value) {
                    debugPrint('Heart Rate notification received: ${value.map((b) =>
                        '0x${b.toRadixString(16).padLeft(2, '0')}}').join(', ')}');
                    _processHeartRate(value);
                },
                onError: (e) => debugPrint('Heart rate notification error: $e'),
            );
        } catch (e) {
            debugPrint('Error setting up heart rate notifications: $e');
        }
    }
}

```

```

    } else {
        debugPrint('Heart Rate characteristic does not support notify');
    }
} catch (e) {
    debugPrint('Heart rate complete setup error: $e');
}

// Setup Battery
try {
    debugPrint('Setting up Battery service...');
    final batteryService = services.firstWhere(
        (s) => s.uuid.toString().toLowerCase() ==
        _batteryServiceUuid.toLowerCase(),
        orElse: () => throw Exception('Battery service not found'),
    );
    debugPrint('Found Battery service: ${batteryService.uuid}');

    final batteryChar = batteryService.characteristics.firstWhere(
        (c) => c.uuid.toString().toLowerCase() ==
        _batteryCharUuid.toLowerCase(),
        orElse: () => throw Exception('Battery characteristic not found'),
    );
    debugPrint('Found Battery characteristic: ${batteryChar.uuid}');
    debugPrint('Battery Properties: read=${batteryChar.properties.read}, '
        'notify=${batteryChar.properties.notify}');

    // Try initial read
    if (batteryChar.properties.read) {
        try {
            final value = await batteryChar.read();
            debugPrint('Initial battery read: ${value.map((b) =>
                '0x${b.toRadixString(16).padLeft(2, '0')}}').join(', ')}');
            _processBattery(value);
        } catch (e) {
            debugPrint('Error reading initial battery: $e');
        }
    }

    // Enable notifications
    if (batteryChar.properties.notify) {
        try {
            final success = await batteryChar.setNotifyValue(true);
            debugPrint('Battery notifications enabled: $success');
            _batterySubscription = batteryChar.lastValueStream.listen(
                (value) {
                    debugPrint('Battery notification received: ${value.map((b) =>
                        '0x${b.toRadixString(16).padLeft(2, '0')}}').join(', ')}');
                    _processBattery(value);
                },
                onError: (e) => debugPrint('Battery notification error: $e'),
            );
        } catch (e) {
            debugPrint('Error setting up battery notifications: $e');
        }
    }
}

```

```

        } else {
            debugPrint('Battery characteristic does not support notify');
        }
    } catch (e) {
        debugPrint('Battery complete setup error: $e');
    }

    _stateStreamController.add(true);
} catch (e) {
    debugPrint('Start error: $e');
    _stateStreamController.add(false);
    rethrow;
}
}

Future<void> stop() async {
    debugPrint('Stopping BLE services...');
    await _accelSubscription?.cancel();
    await _heartRateSubscription?.cancel();
    await _batterySubscription?.cancel();
    _accelSubscription = null;
    _heartRateSubscription = null;
    _batterySubscription = null;
    _stateStreamController.add(false);
    _lastHeartRate = null;
    _lastBatteryLevel = null;
    debugPrint('All BLE services stopped');
}

void dispose() {
    debugPrint('Disposing BLE services...');
    stop();
    _dataStreamController.close();
    _stateStreamController.close();
    debugPrint('BLE services disposed');
}
}
// --- End of lib\accelerometer_service.dart ---

// --- Start of lib\main.dart ---
import 'dart:async';
import 'package:flutter/material.dart';
import 'package:flutter_blue_plus/flutter_blue_plus.dart';
import 'package:permission_handler/permission_handler.dart';
import 'accelerometer_service.dart';

void main() {
    WidgetsFlutterBinding.ensureInitialized();
    FlutterBluePlus.setLogLevel(LogLevel.verbose, color: true);
    runApp(const MyApp());
}

class MyApp extends StatelessWidget {
    const MyApp({Key? key}) : super(key: key);

```

```

@override
Widget build(BuildContext context) {
  return MaterialApp(
    debugShowCheckedModeBanner: false,
    title: 'CL837 Sensor Display',
    theme: ThemeData(
      primarySwatch: Colors.blue,
      visualDensity: VisualDensity.adaptivePlatformDensity,
    ),
    home: const SensorDisplayPage(),
  );
}

class SensorDisplayPage extends StatefulWidget {
  const SensorDisplayPage({Key? key}) : super(key: key);

  @override
  State<SensorDisplayPage> createState() => _SensorDisplayPageState();
}

class _SensorDisplayPageState extends State<SensorDisplayPage> {
  final SensorService _sensorService = SensorService();
  static const String targetDeviceName = 'CL837-0753644';
  BluetoothDevice? connectedDevice;
  SensorData? latestData;
  bool isScanning = false;
  bool isConnecting = false;
  late StreamSubscription<SensorData> _dataSubscription;

  @override
  void initState() {
    super.initState();
    _initializeBluetooth();
  }

  Future<void> _initializeBluetooth() async {
    await _requestPermissions();
    await FlutterBluePlus.turnOn();
  }

  Future<void> _requestPermissions() async {
    await Future.wait([
      Permission.bluetooth.request(),
      Permission.bluetoothScan.request(),
      Permission.bluetoothConnect.request(),
      Permission.location.request(),
    ]);
  }

  Widget _buildStatusRow(String label, dynamic value, IconData icon, {Color?
color}) {
    return Card(

```



```

elevation: 2,
child: Padding(
  padding: const EdgeInsets.all(12.0),
  child: Row(
    children: [
      Icon(icon, size: 24, color: color),
      const SizedBox(width: 12),
      Text(
        label,
        style: const TextStyle(
          fontSize: 16,
          fontWeight: FontWeight.bold,
        ),
      ),
      const Spacer(),
      Text(
        value?.toString() ?? 'N/A',
        style: TextStyle(
          fontSize: 16,
          color: color,
          fontWeight: FontWeight.bold,
        ),
      ),
    ],
  ),
),
);
}

```

```

Widget _buildAxisRow(String axis, double value, Color color) {
  return Card(
    elevation: 2,
    child: Padding(
      padding: const EdgeInsets.all(12.0),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Row(
            children: [
              Container(
                width: 24,
                height: 24,
                decoration: BoxDecoration(
                  color: color.withOpacity(0.2),
                  borderRadius: BorderRadius.circular(4),
                ),
              child: Center(
                child: Text(
                  axis,
                  style: TextStyle(
                    color: color,
                    fontWeight: FontWeight.bold,
                  ),
                ),
              ),
            ],
          ),
        ],
      ),
    ),
  );
}

```

```

    ),
  ),
  const SizedBox(width: 12),
  Expanded(
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        LinearProgressIndicator(
          value: (value + 8.0) / 16.0, // Normalize from -8g to
+8g
          backgroundColor: color.withOpacity(0.1),
          valueColor: AlwaysStoppedAnimation<Color>(color),
        ),
        const SizedBox(height: 4),
        Text(
          '${value.toStringAsFixed(3)} g',
          style: TextStyle(
            color: color,
            fontWeight: FontWeight.w500,
          ),
        ),
      ],
    ),
  ),
),
],
),
],
),
],
),
);
}

Widget _buildDataCard() {
  return Padding(
    padding: const EdgeInsets.all(16.0),
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.stretch,
      children: [
        if (latestData != null) ...[
          _buildStatusRow(
            'Heart Rate',
            latestData!.heartRate != null ? '${latestData!.heartRate} BPM' :
'N/A',
            Icons.favorite,
            color: Colors.red,
          ),
          const SizedBox(height: 8),
          _buildStatusRow(
            'Battery',
            latestData!.batteryLevel != null ? '${latestData!.batteryLevel}%'
: 'N/A',
            Icons.battery_full,
            color: _getBatteryColor(latestData!.batteryLevel),
          ),
        ],
      ],
    ),
  ),

```

```

const SizedBox(height: 16),
const Text(
  'Accelerometer',
  style: TextStyle(
    fontSize: 18,
    fontWeight: FontWeight.bold,
  ),
),
const SizedBox(height: 8),
_buildAxisRow('X', latestData!.x, Colors.red),
const SizedBox(height: 8),
_buildAxisRow('Y', latestData!.y, Colors.green),
const SizedBox(height: 8),
_buildAxisRow('Z', latestData!.z, Colors.blue),
] else
const Center(
  child: Text(
    'Waiting for data...',
    style: TextStyle(
      fontSize: 16,
      color: Colors.grey,
    ),
  ),
),
],
),
);
}

Color _getBatteryColor(int? level) {
  if (level == null) return Colors.grey;
  if (level > 60) return Colors.green;
  if (level > 30) return Colors.orange;
  return Colors.red;
}

Future<void> startScan() async {
  if (isScanning) return;
  setState(() {
    isScanning = true;
  });
  try {
    await FlutterBluePlus.startScan(timeout: const Duration(seconds: 5));
    FlutterBluePlus.scanResults.listen((results) {
      for (ScanResult r in results) {
        debugPrint('Found device: ${r.device.name}');
        if (r.device.name == targetDeviceName) {
          connectToDevice(r.device);
          FlutterBluePlus.stopScan();
          break;
        }
      }
    });
  } catch {}
  await Future.delayed(const Duration(seconds: 5));
}

```

```

        if (mounted) {
            setState(() {
                isScanning = false;
            });
        }
    } catch (e) {
        debugPrint('Error during scan: $e');
        if (mounted) {
            setState(() {
                isScanning = false;
            });
        }

        showError('Error during scan: $e');
    }
}

Future<void> connectToDevice(BluetoothDevice device) async {
    if (isConnecting) return;
    setState(() {
        isConnecting = true;
    });
    try {
        await device.connect(timeout: const Duration(seconds: 10));
        await _sensorService.start(device);
        _dataSubscription = _sensorService.dataStream.listen((data) {
            setState(() {
                latestData = data;
            });
        });
        if (mounted) {
            setState(() {
                connectedDevice = device;
                isConnecting = false;
            });
        }
    } catch (e) {
        debugPrint('Error during connection: $e');
        if (mounted) {
            setState(() {
                isConnecting = false;
            });
        }
        showError('Error during connection: $e');
    }
}

Future<void> disconnectDevice() async {
    try {
        await _dataSubscription.cancel();
        await _sensorService.stop();
        await connectedDevice?.disconnect();
    } catch (e) {
        debugPrint('Error during disconnect: $e');
    }
}

```

```

    }
    if (mounted) {
      setState(() {
        connectedDevice = null;
        latestData = null;
      });
    }
  }
}

void showError(String message) {
  if (!mounted) return;
  ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(
      content: Text(message),
      backgroundColor: Colors.red,
      duration: const Duration(seconds: 3),
    ),
  );
}

@override
void dispose() {
  _dataSubscription.cancel();
  _sensorService.dispose();
  disconnectDevice();
  super.dispose();
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('CL837 Sensor Display'),
      actions: [
        if (connectedDevice != null)
          IconButton(
            icon: const Icon(Icons.bluetooth_connected),
            tooltip: 'Disconnect',
            onPressed: disconnectDevice,
          ),
      ],
    ),
    body: Column(
      children: [
        Container(
          color: Colors.grey[100],
          padding: const EdgeInsets.all(16.0),
          child: Row(
            children: [
              Expanded(
                child: ElevatedButton.icon(
                  icon: const Icon(Icons.search),
                  label: Text(
                    isScanning ? 'Scanning...' :

```

```

        isConnecting ? 'Connecting...' :
        'Scan for Device'
      ),
      onPressed: (isScanning || isConnecting) ? null : startScan,
    ),
  ],
),
),
if (connectedDevice != null)
  Expanded(
    child: SingleChildScrollView(
      child: _buildDataCard(),
    ),
  ),
],
),
);
}
}
// --- End of lib\main.dart ---

```